

Урок 22. Базы данных. Типы, виды. Создание БД. Часть 3.

ФИО _____

Тест

- 1) *Что из перечисленного **лучше всего** описывает назначение базы данных?*
 - A) Хранить данные без структуры, как набор файлов
 - B) Централизованно хранить данные по правилам, обеспечивая целостность и поиск
 - C) Только создавать красивые отчёты
 - D) Только ускорять интернет
- 2) *Что такое **СУБД**?*
 - A) Язык SQL
 - B) Таблица в базе данных
 - C) Программное обеспечение для создания/хранения/изменения/анализа данных
 - D) Формат JSON
- 3) *Чем отличается **БД** от обычного набора файлов? Напиши 2 отличия.*

- 4) *В реляционной базе данных данные обычно хранятся:*
 - A) В виде связанных таблиц (строки и столбцы)
 - B) В виде дерева папок
 - C) В виде графа
 - D) В виде одного JSON-файла
- 5) *Что обычно является уникальным идентификатором строки в таблице?*
 - A) Любой столбец
 - B) Только текстовое поле
 - C) Ключ (primary key)
 - D) Название таблицы
- 6) *Что такое **SQLite**?*
 - A) Серверная СУБД, требующая отдельного процесса сервера
 - B) Библиотека для FastAPI
 - C) Язык запросов
 - D) Встраиваемая СУБД, где база хранится в одном файле
- 7) *Что делает строка `engine = db.create_engine("sqlite:///myDatabase.db", future=True)`?*
 - A) Создаёт «движок» — объект для управления подключением и выполнением SQL-команд
 - B) Создаёт таблицу books
 - C) Выполняет SELECT
 - D) Создаёт JSON-файл
- 8) *Для чего нужны `metadata = db.MetaData()` и `metadata.create_all(engine)`?*
 - A) Чтобы распечатать строки таблицы

- В) Чтобы описать структуру таблиц и затем создать их в базе
 С) Чтобы обновить рп
 D) Чтобы включить документацию API
 9) Объясни, что делает `.where(...)` в запросе `SELECT` и приведи пример условия своими словами.

10) Что верно про `SQLAlchemy Core` и `ORM`?

- A) Core — это ORM, а ORM — это SQL
 B) Core нужен только для FastAPI
 C) ORM работает только с SQLite
 D) Core помогает строить запросы как объекты Python (абстракция над SQL), ORM позволяет работать через классы-объекты

Практическая работа:

Переходим к ранее написанному нами `students-API`. Наша задача внедрить в него БД. В чем разница между тем, чтобы использовать БД без API и внутри API мы обсуждали на прошлом уроке.

1. В папке проекта создаем нашу базу данных со следующим кодом:

```
import sqlalchemy as db
from pathlib import Path

# БД будет лежать в myfirstapi/data/students.db
DB_PATH = Path(__file__).resolve().parents[2] / "data" / "students.db"
DB_PATH.parent.mkdir(parents=True, exist_ok=True)

engine = db.create_engine(f"sqlite://{DB_PATH}", future=True)
metadata = db.MetaData()

students = db.Table(
    "students", metadata,
    db.Column("student_id", db.Integer, primary_key=True),
    db.Column("first_name", db.Text, nullable=False),
    db.Column("last_name", db.Text, nullable=False),
    db.Column("date_of_birth", db.Text, nullable=False),
    db.Column("email", db.Text, nullable=False),
    db.Column("phone_number", db.Text, nullable=False),
    db.Column("address", db.Text, nullable=False),
    db.Column("enrollment_year", db.Integer, nullable=False),
    db.Column("grade", db.Integer, nullable=False),
    db.Column("special_notes", db.Text, nullable=True),
)

def init_db() -> None:
    metadata.create_all(engine)
```

Часть из этого мы уже реализовали на прошлом уроке, можно скопировать.

2. В schemas.py оставляем как есть (только убедись, что StudentUpdate импортируется в app.py)
3. Переписать src/myproject/app.py на работу с БД: убираем json_to_dict_list / dict_list_to_json, вместо них используем engine и students.

```
from fastapi import FastAPI, HTTPException, Query, Path as PathParam, Response
import sqlalchemy as db

from myproject.schemas import Student, StudentUpdate, Error
from myproject.db_core import engine, students, init_db

app = FastAPI(
    title="School API (SQLite)",
    description="Мини-API для списка учеников на SQLite.",
    version="2.0.0",
    docs_url="/docs",
    redoc_url=None,
)

app.openapi_tags = [
    {"name": "health", "description": "Проверка, что сервер жив"},
    {"name": "students", "description": "Эндпоинты по ученикам"},
]

@app.on_event("startup")
def startup():
    init_db()

@app.get("/", tags=["health"])
def home_page():
    return {"message": "Привет, Мир!"}
```

В начале без глобальных изменений.

4. Далее смотрим на код sqlalchemy с прошлого урока и добавляем его в каждый запрос, адаптируя его, начнем с GET (SELECT в SQL):

```
@app.get(
    ...
)
def get_all_students(
    grade: int | None = Query(None, ge=1, le=11),
):
    with engine.begin() as conn:
        stmt = db.select(students).order_by(students.c.student_id)
        if grade is not None:
            stmt = stmt.where(students.c.grade == grade)
        rows = conn.execute(stmt).fetchall()
    return [dict(r._mapping) for r in rows]

@app.get(
    ...
)
def get_students_by_grade(
    grade: int = PathParam(..., ge=1, le=11),
    last_name: str | None = Query(None),
):
    with engine.begin() as conn:
        stmt = db.select(students).where(students.c.grade == grade)
```

```

        if last_name:
            ln = last_name.strip()
            stmt = stmt.where(db.func.lower(students.c.last_name) ==
db.func.lower(ln))

        stmt = stmt.order_by(students.c.student_id)
        rows = conn.execute(stmt).fetchall()

    return [dict(r._mapping) for r in rows]

```

Делаем **with engine.begin() as conn:** внутри каждого метода. Чтобы вывести результат используем списковое включение формата `[dict(r._mapping) for r in rows]`.

5. POST-запрос (INSERT)

```

@app.post(
    ...
)
def create_student(payload: Student):
    with engine.begin() as conn:
        # проверка на существование ID
        exists = conn.execute(
            db.select(students.c.student_id).where(students.c.student_id ==
payload.student_id)
        ).fetchone()
        if exists is not None:
            raise HTTPException(status_code=409, detail="student_id already
exists")

        conn.execute(db.insert(students), [payload.model_dump()])

    return payload

```

6. PUT-запрос (UPDATE)

```

@app.put(
    ...
)
def replace_student(student_id: int, payload: Student):
    if payload.student_id != student_id:
        raise HTTPException(status_code=400, detail="student_id in path and
body must match")

    with engine.begin() as conn:
        result = conn.execute(
            db.update(students)
            .where(students.c.student_id == student_id)
            .values(**payload.model_dump(exclude={"student_id"}))
        )
        if result.rowcount == 0:
            raise HTTPException(status_code=404, detail="student not found")

    return payload

```

7. PATCH (UPDATE) и DELETE попробуйте самостоятельно.